

08 — Usage Guide

Prerequisites

- **Python 3.10 or later**
- **llvmlite** (`pip install llvmlite`)
- **MinGW gcc** (for native binary output) — already at `C:\mingw64\bin\gcc.exe`

Verify your setup:

```
python --version          # 3.10+
python -c "import llvmlite; print('ok')"
```

```
gcc --version             # MinGW-W64 13.2.0 or similar
```

First-Time Setup

Build the C stdlib (needed only once, or after editing `stdlib/forensics.c`):

```
cd "JOCKY Language\JOCKY"
python build_stdlib.py
```

Expected output:

```
[build_stdlib] Compiling ...stdlib\forensics.c
                gcc: C:\mingw64\bin\gcc.EXE

OK - ...stdlib\forensics.o (21098 bytes)

Stdlib is ready.
```

Compiler Command Reference

```
python compiler.py <source.jk> [options]
```

Options

Flag	Effect
<code>--run</code>	JIT-execute immediately. No output files. Uses Python stdlib callbacks.

Flag	Effect
<code>--emit-ir</code>	Write LLVM IR to <code>output/<name>.ll</code> . Combine with <code>--run</code> or default.
<code>--no-obfuscate</code>	Skip all obfuscation passes. Strings stay readable. Still produces <code>.o</code> / <code>.exe</code> .
<code>-o <dir></code>	Output directory (default: <code>output/</code>).

Mode combinations

```
# Quick test (JIT, no files written):
python compiler.py tests/hello.jk --run

# JIT + see the IR:
python compiler.py tests/hello.jk --run --emit-ir

# Native exe (with obfuscation, encrypted strings):
python compiler.py tests/hello.jk

# Native exe (no obfuscation, readable strings, functional binary):
python compiler.py tests/hello.jk --no-obfuscate

# Inspect what the compiler generates (IR only, no execution):
python compiler.py tests/hello.jk --emit-ir --no-obfuscate
# Then: notepad output\hello.ll
```

Running the Tests

All four test programs should be verified in order.

Hello world

```
python compiler.py tests/hello.jk --run
```

Expected:

```
[JOCKY] Hello from JOCKY!
[JOCKY] JOCKY compiler is working.
```

Basic language features

```
python compiler.py tests/test_basic.jk --run
```

Tests: variable declarations, arithmetic, conditionals, boolean logic, loops, break.

Functions

```
python compiler.py tests/test_functions.jk --run
```

Tests: user-defined functions, parameters, return values, nested function calls.

Forensics demo (SIH)

```
python compiler.py tests/demo_forensics.jk --run
```

Expected:

```
[JOCKY] JOCKY Forensic Scanner
[JOCKY] SIH Hackathon – Component 1 Demo
[JOCKY] Initialising process scan...
[JOCKY] Process scan complete
[JOCKY] svchost.exe – detected
[JOCKY] explorer.exe – detected
[JOCKY] malware.exe – not present
[JOCKY] Forensic scan finished
```

Native binary test

```
python compiler.py tests/hello.jk --no-obfuscate
.\output\hello.exe
```

Expected: same output as JIT mode.

Writing Your Own JOCKY Program

Step 1: Create a `.jk` file

```
## myscript.jk
## My first JOCKY script

func start() -> nothing {
    report(`My program runs!`)
}
```

Save as `tests/myscript.jk` (or anywhere inside the JOCKY folder).

Step 2: Run it

```
python compiler.py tests/myscript.jk --run
```

Step 3: Iterate

Make changes, re-run. The compiler gives clear error messages:

```
[1/5] Lexer – tokenising source...  
      OK – 14 tokens  
[2/5] Parser – building AST...  
      Parse error at line 3: Expected RPAREN, got IDENTIFIER ("x")
```

Writing a Process Scanner

```
## scanner.jk  
  
func scan(target : text) -> flag {  
  var procs : raw := procs_list()  
  var total : num := proc_count(procs)  
  var i      : num := 0  
  
  loop (i lt total) {  
    var name : text := proc_name(procs, i)  
    check (name is target) {  
      give yes  
    }  
    i <- i + 1  
  }  
  give no  
}  
  
func start() -> nothing {  
  report(`Starting scan`)  
  
  check (scan(`svchost.exe`)) {  
    report(`svchost.exe: RUNNING`)  
  } otherwise {  
    report(`svchost.exe: not found`)  
  }  
  
  check (scan(`malware.exe`)) {  
    report(`ALERT: malware.exe found!`)  
  } otherwise {  
    report(`System clean`)  
  }  
}
```

```
}  
  
report(`Scan complete`)  
}
```

```
python compiler.py tests/scanner.jk --run
```

SIH Demo Walkthrough

This sequence demonstrates all three AV-evasion features in order.

Step 1: Show the source is unrecognisable

Open `tests/demo_forensics.jk` in a text editor. Point out:

- Keywords like `check`, `loop`, `give`, `func`, `also`, `procs_list`
- No Python `import`, no PowerShell `$`, no VBScript `Dim`
- AV script parsers cannot parse this — unknown grammar

Step 2: Run it and show it works

```
python compiler.py tests/demo_forensics.jk --run
```

Show the output — `svchost` detected, `explorer` detected, `malware` not found. The forensic logic is correct.

Step 3: Show the IR with plaintext strings (no obfuscation)

```
python compiler.py tests/demo_forensics.jk --emit-ir --no-obfuscate
```

Open `output/demo_forensics.ll` and scroll to the bottom. Show the string globals:

```
@".jk_str.0" = internal constant [21 x i8] c"JOCKY Forensic Scanner\00"  
@".jk_str.1" = internal constant [14 x i8] c"svchost.exe\00"
```

Point out: these strings are visible in the binary. A string scanner would find them.

Step 4: Show the IR with encrypted strings (with obfuscation)

```
python compiler.py tests/demo_forensics.jk --emit-ir
```

Open the new `output/demo_forensics.ll`. Show:

```
@".jk_str.0"      = internal constant [21 x i8] c"\3f\1a\7b..."
@".jk_str.0.key"  = internal constant [1 x i8]  c"J"
@_jocky_build_id = internal constant [16 x i8]  c"\a3\f1\2c..."
@_jocky_entropy  = internal constant i64 7284619203847561829
```

Point out: no readable strings. Random encrypted bytes. `_jocky_build_id` is different each time.

Step 5: Prove polymorphism with different SHA-256

Run twice, show different hashes:

```
python compiler.py tests/demo_forensics.jk
# SHA-256: abc123...

python compiler.py tests/demo_forensics.jk
# SHA-256: def456... ← completely different
```

Conclusion: "The same source code produces a different binary on every compile. AV hash databases cannot blacklist it — the hash changes before any AV vendor can catalogue it."

Step 6: Run the native binary (no Python needed)

```
python compiler.py tests/demo_forensics.jk --no-obfuscate
.\output\demo_forensics.exe
```

Point out: this `.exe` runs without Python, without LLVM, without any runtime. Pure native machine code.

Inspecting Compiler Output

The `.ll` IR file

```
python compiler.py tests/hello.jk --emit-ir --no-obfuscate
```

Opens `output/hello.ll`. Sections to look at:

- `declare` lines at the top — external `stdlib` functions
- `define void @start()` — the compiled JOCKY function
- `@".jk_str.*"` globals at the bottom — string literals
- `getelementptr` instructions — how string pointers are computed
- `call void @report(...)` — function calls

Verifying the build in binary mode

```
python compiler.py tests/hello.jk
```

Output includes:

- Object file path and size
- SHA-256 of the object file
- Executable path and size
- SHA-256 of the `.exe`
- Command to run the exe

Error Reference

Lexer errors

Error message	Cause	Fix
unexpected character '''	Used double-quotes instead of backticks	Change "text" to `text`
unterminated string literal	Missing closing backtick	Add ` at end of string
unexpected character '@'	@ is not a JOCKY operator	Remove it

Parse errors

Error message	Cause	Fix
Expected ARROW_R (->), got LBRACE	Missing -> type in function definition	Add -> nothing before {
Expected ASSIGN (:=), got EOF	Variable declared without value	Add := value
Expected RPAREN, got IDENTIFIER	Missing) to close condition	Add) after condition
Expected 'func', got IDENTIFIER	Top-level code outside a function	Wrap in func start() -> nothing { }

Semantic errors

Error message	Cause	Fix
Undefined variable 'x'	Using a variable before declaring it	Add var x : type := ... before use

Error message	Cause	Fix
Undefined function 'foo'	Calling a function that doesn't exist	Check spelling; must be in same file
Type mismatch: expected 'num', got 'text'	Wrong type in assignment or argument	Match types; use correct function arg
Function 'report' expects 1 argument, got 0	Wrong argument count	Pass the required arguments
'check' condition must be 'flag'	Non-boolean condition	Use a comparison: <code>check (x > 0)</code>

Linker errors

Error	Cause	Fix
forensics.o not found	<code>build_stdlib.py</code> not run	<code>python build_stdlib.py</code>
undefined reference to 'start'	No <code>func start()</code> in source	Add entry function
undefined reference to 'report'	forensics.o missing or corrupted	Re-run <code>build_stdlib.py</code>

Project Structure Quick Reference

```

JOCKY/
├─ compiler.py      python compiler.py <file.jk> [--run] [--emit-ir] [--no-
obfuscate]
├─ build_stdlib.py  python build_stdlib.py (run once)
├─ jocky/           compiler internals (do not run directly)
├─ stdlib/
│   └─ forensics.c  edit to add real Windows API implementations
│       └─ forensics.o compiled by build_stdlib.py
├─ tests/           example .jk programs
└─ output/          compiled output (.o, .exe, .ll)

```

Quick Command Reference

```

## Setup (once)
python build_stdlib.py

## Run a script (JIT, no output files)
python compiler.py tests/hello.jk --run

## Compile to native exe
python compiler.py tests/hello.jk --no-obfuscate
.\output\hello.exe

```



```
## Compile with obfuscation (AV demo)
python compiler.py tests/hello.jk
## then compare SHA-256 between two runs

## See the LLVM IR
python compiler.py tests/hello.jk --emit-ir --no-obfuscate
## open output/hello.ll

## Run the SIH demo
python compiler.py tests/demo_forensics.jk --run

## Full demo pipeline
python compiler.py tests/demo_forensics.jk --emit-ir --no-obfuscate
python compiler.py tests/demo_forensics.jk --emit-ir
.\output\demo_forensics.exe  ## (after build with --no-obfuscate)
```